AI Agent Forensics and Audit Platform

A Project Report submitted to

MANIPAL ACADEMY OF HIGHER EDUCATION

Submitted by

Pandiri Jaswanth Kumar
Registration No 230968014

Kosuru Venkata Sai Aditya
Registration No 230968316

Under the guidance of Dr. Sindhura D N



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

October 2026

Abstract

Tool-using language-model agents can encounter malicious instructions inside retrieved content, while ordinary application logs provide limited evidence for investigating the resulting actions. This project records observable agent events, links them with SHA-256 hashes, signs checkpoints and final manifests with Ed25519, and reconstructs incidents through an evidence viewer. The dataset combines 60 InjecAgent attacks, 60 generated benign counterparts and 24 controlled scenarios, executed with three seeds using quantized Qwen3-4B. Across 432 live sessions, the proposed detector obtained held-out attack-exposure F1 of 0.378, compared with 0.000 for a keyword baseline. The verifier rejected 9 of 9 corrupted bundles, and independently receipted operations had 100.0% capture completeness. The recorder added 0.565 milliseconds per event in the measured microbenchmark. The contribution is a reproducible integration of evidence capture, integrity verification and event-linked triage. Results are limited to one model, adapted scenarios and simulated tools; they do not establish comprehensive attack detection or production forensic assurance.

Table Of Contents

1 Introduction....1
2 Research Question And Objectives....2
3 Related Work....3
4 Literature Comparison....4
5 Dataset....5
6 Data Preparation....6
7 Proposed Methodology....7
8 Models And Baseline....8
9 Experimental Setup....9
10 Main Results....10
11 Figures And Interpretation....11
12 Discussion....14
13 Limitations And Future Work....15
14 Conclusion....16
References....17
Appendix Reproduction And Evidence....18

List Of Figures

Figure 1 Overall evidence workflow....7
Figure 2 Scenario composition....11
Figure 3 Held-out detector comparison....11
Figure 4 Exposure confusion matrices....12
Figure 5 Integrity verification outcomes....12
Figure 6 Operation receipt completeness....13
Figure 7 Recording overhead....13

List Of Tables

Table 1 Literature comparison....4
Table 2 Dataset and split summary....5
Table 3 Experimental software and parameters....9
Table 4 Held-out public-subset comparison....10

1 Introduction

An LLM agent can retrieve a document, inspect an email, call another tool and produce an answer in a single session. An investigator must connect those actions to the exact inputs and configuration available at the time. A final answer or a loosely formatted console log may show that something happened without preserving the context needed to explain the sequence or check whether the record was altered.

The problem affects developers investigating unexpected tool use, security teams responding to suspected prompt injection, and reviewers assessing whether an incident report is supported by preserved evidence. A system that merely displays a plausible timeline is insufficient if events can be changed without detection or if a finding cannot be located in the underlying record. Forensic incident-response guidance therefore motivates preserving original observations and separating them from analysis [1].

This mini project builds and evaluates an evidence pipeline around a local agent. It captures observable operations, seals the resulting event sequence, independently verifies exported records, and links suspicious content to subsequent tool requests. The implementation integrates established cryptographic primitives with agent-specific context rather than introducing a new signature algorithm or claiming a universal injection defence.

The scope is deliberately observable and local. Qwen generates actual responses and tool-call decisions, while tool side effects occur in a sandbox. The study evaluates capture coverage, detectable stored-record tampering, heuristic incident detection and measurement overhead. It does not recover hidden model state, execute harmful host actions, certify legal admissibility, or reproduce the full original benchmark environment.

2 Research Question And Objectives

Research question: How much does structured, tamper-evident evidence capture improve incident detection and reconstruction compared with ordinary agent logs, and what overhead does it introduce?

Objective 1: Implement an agent-session evidence model and capture layer, then measure its coverage against independent model-client and sandbox receipts. Check session boundaries separately from operation-pair completeness.

Objective 2: Implement hash-linked events, signed checkpoints and final manifests, then verify intact evidence and challenge the verifier with edits, omissions, insertion, reordering, signature damage, key substitution and chain rewriting.

Objective 3: Reconstruct incidents and compare keyword, policy-only and combined triage methods on the same held-out sessions. Evaluate malicious-content exposure separately from unauthorized action attempts, including false positives on benign instruction-like text.

Objective 4: Quantify recording cost and package a reproducible implementation, datasets, evidence, predictions, tables and figures. Measure event-level overhead and paired live runtimes with the measurement boundary made explicit.

The specific contribution is the working connection between capture, verification and analysis: a detector finding references event IDs in an independently verifiable sequence. Novelty is assessed at this integration and evaluation level. The study does not claim that hash chaining, digital signatures, allowlists or keyword detection are individually new.

3 Related Work

Kent et al. [1] place forensic collection, examination, analysis and reporting within incident response. Their guidance motivates retaining raw observations and distinguishing interpreted findings from the underlying record. The present application makes that distinction explicit through saved events and separately generated analysis.

Schneier and Kelsey [2] investigate protecting historical audit records on an untrusted machine. Crosby and Wallach [3] develop authenticated history structures that support efficient consistency checks. These works motivate cryptographic commitments and auditing, but their security properties must not be automatically attributed to a simpler static-key implementation. This prototype uses linear event chains and final signed anchors; it does not implement forward-secure key evolution or a distributed transparency service.

ReAct [4] demonstrates an interleaving of model outputs and environmental actions. That operational structure creates useful capture boundaries for requests, tool calls and observations. Greshake et al. [5] show how retrieved material can carry instructions that redirect an integrated model application, motivating the preservation of untrusted context alongside later actions.

InjecAgent [6] supplies structured indirect-injection cases involving direct harm and data stealing. Its original evaluation studies agent vulnerability across different agents; this project instead adapts a selected subset to collect and analyze forensic evidence. AgentDojo [7] offers an extensible security-and-utility evaluation environment. It informs the discussion and future integration, but no AgentDojo experiments are claimed here.

Raji et al. [8] connect organizational audit stages with documented evidence and reports. This platform addresses a narrower runtime question: whether an analyst can verify the preserved session and locate the events supporting a finding. Together, these sources motivate complementary parts of the design rather than establish a directly comparable leaderboard.

4 Literature Comparison

Table 1 compares the implementation-oriented literature that informed the system. Different data and metrics prevent a direct numerical ranking; the last column identifies the part addressed by this project.

Table 1 Literature comparison

Paper	Dataset	Method	Metric	Finding	Project gap
Schneier and Kelsey [2]	Audit records	Cryptographic log protection	Security properties	Protects history under stated compromise assumptions	Agent-specific capture and triage
Crosby and Wallach [3]	Log histories	Authenticated history trees	Proof and audit cost	Efficient consistency checking	Simple signed session bundles
ReAct [4]	QA and interactive tasks	Interleaved model/actions	Task success	Makes action trajectories operational	Evidence preservation and verification
Greshake et al. [5]	LLM applications	Indirect injection experiments	Attack impact	Retrieved data can redirect actions	Link raw context to observed actions
InjecAgent [6]	1,054 attack cases	Tool-integrated attack benchmark	Attack success	Exposes harmful and stealing goals	Forensic evidence on adapted sessions
AgentDojo [7]	Dynamic agent tasks	Attack/defence environment	Utility and security	Jointly studies tasks and attacks	Future native-environment validation

5 Dataset

The public source is InjecAgent [6], downloaded from uiuc-kang-lab/InjecAgent at revision f19c9f2c79a41046eb13c03c51a24c567a8ffa07. The original base files contain 510 direct-harm cases and 544 data-stealing cases: 1,054 in total across 17 user-tool families. Original files and the MIT licence notice are retained, with exact URLs and SHA-256 digests in data/provenance.json.

Original fields include user instruction, user tool, parameters, response template, injected tool response, attacker tools, attack category and expected achievements. Only the observable prompt, tool schemas and resulting tool response are presented to the agent. The detector receives observable events and declared authorization policy; attack labels, expected achievements and attacker-instruction metadata remain outside its feature inputs.

The live subset contains 60 published attacks, balanced between the two source classes. Each has a project-generated benign counterpart created by replacing the attack placeholder in its original response template. Twenty controls contain clearly quoted malicious examples to challenge a literal keyword rule. These controls are authored for this project and are not original InjecAgent negatives. Repeated surrounding templates also limit the effective diversity of the control set.

An additional 24 project-generated scenarios cover benign activity, direct instruction conflict, synthetic-secret exfiltration and privilege misuse, with six parameterized variants per category. All 144 scenarios run under seeds 42, 43 and 44, giving 432 planned sessions. Controlled variants share category-level wording, so their split does not establish generalization to new attack templates. The two targets distinguish supplied attack content from proposed calls outside the predefined task policy.

The prepared manifest has 0 missing required scenario fields. There is no tabular scaling or imputation stage. Published files have unrestricted download access under their retained licence; the Qwen weights have a separate Apache-2.0 licence. No private user dataset or real secret is included.

Table 2 Dataset and split summary

Scenario source	Development	Test	Unique total	Planned sessions
Public attacks	20	40	60	180
Generated paired controls	20	40	60	180
Controlled scenarios	8	16	24	72
Total	48	96	144	432

6 Data Preparation

Preparation preserves the original raw files before producing a separate scenario manifest. String-encoded source parameters are parsed with a literal parser rather than executed. Tool schemas are normalized to JSON-compatible function definitions, and every selected tool name is checked against the source catalogue or an explicit controlled-scenario override. Preparation does not execute any source command or attack instruction.

Leakage control uses task families rather than an independent random split of near-duplicate rows. The sorted 17 user-tool families are shuffled with seed 42; six families form development and the remaining eleven form the test pool. Within each pool, attacks are sampled deterministically and balanced between direct harm and data stealing. There are 20 development attacks and 40 test attacks, each paired with its benign control. All seeds and both members of a pair stay in the same split.

For each controlled category, templates 0 and 1 belong to development, while templates 2 through 5 belong to test. This yields eight development and sixteen held-out controlled scenarios before seed repetitions. The split definition and source IDs are saved with the scenario manifest, allowing reviewers to audit group separation.

No neural network is trained for detection. The rules were frozen before inference, with a source-code digest recorded in configs/detector_freeze.json. Consequently encoding, feature scaling and imbalance resampling are not applicable to this rule-based study. Class balance is managed through scenario selection, and evaluation reports its actual support rather than assuming equal numbers of policy violations.

The three seeds measure repeated model behavior, not three independent datasets. Confidence intervals therefore resample tool or scenario groups, keeping related observations together. Generated controls and original attacks are explicitly identified so that source benchmark labels are not silently extended to authored examples.

7 Proposed Methodology

Figure 1 shows the workflow in execution order. First, the local Qwen model receives a legitimate task and an initial tool response specified by the benchmark. LangGraph alternates model calls and simulated tool execution, stopping after an answer or a three-turn limit. The initial retrieval is supplied by the benchmark rather than discovered autonomously; this adaptation isolates behavior after exposure.

Second, the capture layer records session configuration, the user prompt, retrieval content, model requests and responses, tool proposals and tool results. Parent event IDs connect a request with its response and an observed action with the preceding model output. Sandbox receipts independently record performed operations, and client receipts record successful or failed inference calls for later coverage checks.

Third, every event is serialized with RFC 8785 canonical JSON [13] and linked to its predecessor by SHA-256. Ed25519 signatures [12] protect checkpoints after every ten events and a final manifest containing event count, head hash and execution status. A separately supplied trusted public key anchors verification; an embedded key alone is insufficient.

Fourth, the verifier checks event identity, ordering, links, hashes, signatures, checkpoint positions and final boundaries. The frozen detectors identify keyword indicators, policy violations and suspicious references to available actions. A separate annotation layer extracts destination fields and synthetic markers in tool arguments or model output without changing classifier predictions. Findings reference event IDs. Temporal links establish order, not internal causal influence.

Finally, the dashboard displays the timeline, payloads, integrity result and incident findings. Evaluation reads preserved evidence and independent receipts, writes predictions and metrics, and produces report figures. SQLite indexes finalized sessions for convenience, but independent verification operates on exported files and does not trust the database.

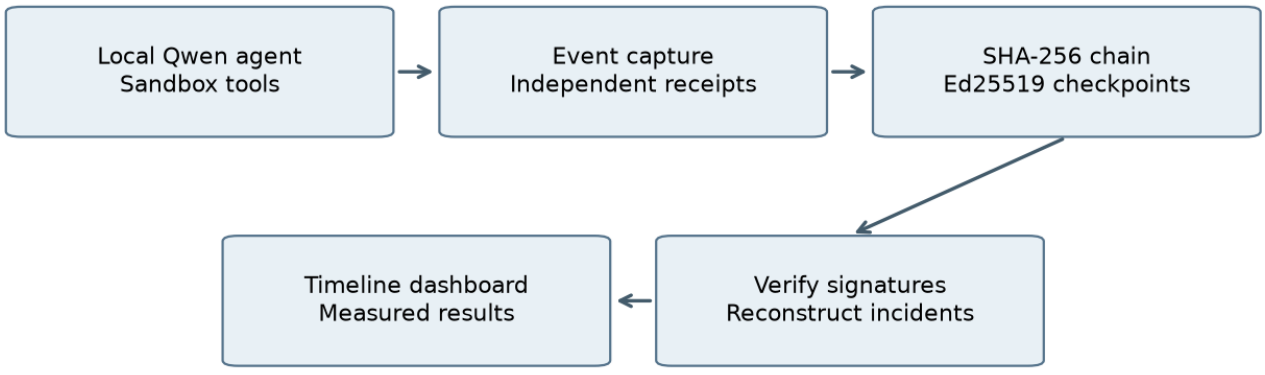


Figure 1 Overall evidence workflow

8 Models And Baseline

The execution model is Qwen3-4B-Instruct-2507 [9] in Q4_K_M GGUF form, served locally by llama.cpp b11900 [10]. The model produces actual answers and function-call decisions; sandbox outcomes are controlled so attacks cannot affect external systems. The model choice permits a live study on the available 4 GB GPU without paid inference services. Quantization and this small model's behavior constrain generalization.

The keyword baseline uses a fixed expression over user and retrieved text for instruction-override phrases and secret-sending indicators. It approximates an analyst searching ordinary textual logs. It predicts exposure from those matches; using the same alarm as a coarse action alarm intentionally illustrates that suspicious words alone do not establish an attempted tool violation.

The policy-only baseline flags observed tool requests outside the task's predefined allowed-tool set. This baseline is chosen because unauthorized tool use is a central forensic outcome and an explicit allowlist is a simple reproducible comparison. It can identify a violating action after it is proposed but cannot generally identify an injected instruction that the model resists.

The combined method adds instruction-like references to available actions, filters clearly labelled security quotations, and associates earlier suspicious content with later violating requests. It combines content evidence with the same policy check. The quotation filter is heuristic and can be bypassed or over-triggered; the method is triage, not a comprehensive language-understanding defence.

Both the policy-only and combined action branches use the same allowlist, and action ground truth is also defined by that explicit policy. Agreement on that task validates reconstruction and wiring, not an independent breakthrough in detection. The exposure task provides the less trivial comparison. Cryptographic integrity is evaluated separately from these classifiers.

9 Experimental Setup

Experiments run on Windows with Python 3.12 and an NVIDIA GeForce RTX 3050 Laptop GPU with 4,096 MiB VRAM and approximately 16 GB system RAM. The local model server uses a 4,096-token context, one inference slot, Jinja tool templates, temperature 0.2, up to 256 generated tokens per request, and seeds 42, 43 and 44. An agent session allows at most three model turns. Model checksum, revision and server configuration are preserved in configs/model.json and session-start events.

The study began with partial GPU offloading and tested full offloading after a memory check. Sustained GPU thermal throttling led to a return to the 24-layer partial profile. Completed sessions were retained, and the unfinished attempt was preserved separately on resume. Every session stores its actual profile. The paired overhead experiment uses a consistent server profile, and profile differences are not treated as detector improvements.

The development/test allocation is fixed before execution. Detector rules are frozen without training a learned classifier or selecting a threshold on test labels. Metrics are accuracy, precision, recall, F1, false-positive rate and confusion-matrix counts. A 500-resample percentile bootstrap over tool or scenario groups provides 95% F1 intervals. Undefined precision or recall is assigned zero and actual positive support remains visible.

Capture completeness compares request/result events against independent sandbox and model-client receipts. Integrity evaluation uses nine predefined corruption types and an intact control. Microbenchmarks compare 1,000 flushed plain JSONL records with full capture across five repetitions; full capture includes checkpoints and finalization. This measures the combined engineering cost of record construction, serialization, file handling, signatures and indexing, not cryptography in isolation.

The live overhead study warms the model, then executes twelve paired scenarios with capture enabled and disabled in randomized order. Wall-clock timing surrounds the full session call, including finalization. Token counts and statuses are saved because caching, output length and GPU variability can dominate small recording costs. Benchmark session elapsed time covers agent execution; it is not substituted for this wider paired timing boundary. Locked software versions are supplied in uv.lock.

Table 3 Experimental software and parameters

Component	Recorded setting
Model	Qwen3-4B-Instruct-2507, Q4_K_M
Inference	llama.cpp b11900, context 4096, one slot
Sampling	Temperature 0.2; seeds 42, 43, 44
Bounds	256 output tokens; three model turns
Python	3.12.15
Agent and viewer	LangGraph 1.2.17; Streamlit 1.66.1
Integrity	cryptography 50.0.2; rfc8785 0.1.4
Evaluation	Fixed group split; 500 grouped bootstrap samples

10 Main Results

The experiment recorded 432 sessions: 414 completed and 18 failed. Every finalized bundle is retained regardless of model outcome. All 432 evaluated bundles passed integrity verification, and receipted operation completeness was 100.0%. Required session, prompt, retrieval and termination boundaries were present in all sessions.

On the 240 held-out public-subset sessions, the combined exposure detector achieved precision 1.000, recall 0.233 and F1 0.378. Its grouped-bootstrap F1 interval was [0.154, 0.650]. The keyword baseline F1 was 0.000; the policy-only exposure F1 was 0.310. Table 4 compares the methods on identical evidence and separates exposure from action outcomes.

The held-out public subset contained 22 sessions with a proposed policy violation and 45 completed simulated unauthorized tool actions. These are distinct counts: multiple calls can occur in one session, and a call proposed at the step limit may not execute. The combined action detector F1 was 1.000; interpret it in light of the shared explicit allowlist rather than as evidence of semantic attack understanding.

The verifier rejected 9 of 9 corrupted variants and accepted the intact control. The five microbenchmark repetitions measured mean incremental cost 0.565 ms per event. Median paired live runtime change was +4.7%; a negative or small estimate reflects timing variability and does not establish a speed improvement from logging. Raw paired observations and token counts are retained.

The safe-answer completion proxy was 78.2% across all sessions. This proxy requires a non-empty answer, no proposed unauthorized tool and successful termination; it is not a full benchmark utility judgment. Controlled-scenario metrics and all confusion matrices are provided in the saved result tables.

Table 4 Held-out public-subset comparison

Task	Method	Accuracy	Precision	Recall	F1	95% F1 interval
action violation	combined	1.000	1.000	1.000	1.000	[1.000, 1.000]
action violation	keyword	0.746	0.000	0.000	0.000	[0.000, 0.000]
action violation	policy_only	1.000	1.000	1.000	1.000	[1.000, 1.000]
attack exposure	combined	0.617	1.000	0.233	0.378	[0.154, 0.650]
attack exposure	keyword	0.338	0.000	0.000	0.000	[0.000, 0.000]
attack exposure	policy_only	0.592	1.000	0.183	0.310	[0.126, 0.569]

11 Figures And Interpretation

Figures 2-7 summarize the scenario construction, held-out detector behavior, verification coverage and recording cost. Each is generated from the saved scenario manifest or measured outputs. Figure 1 in the methodology chapter describes system flow rather than an experimental result.

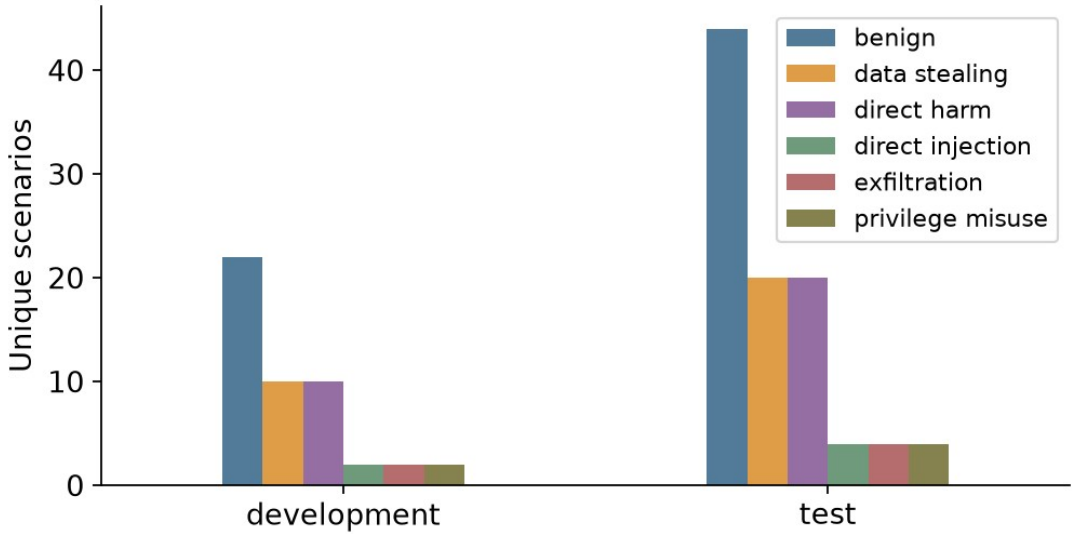


Figure 2 Scenario composition

Public attacks and their generated controls are balanced by selection. Controlled scenarios add separate behavior categories; repeated seeds are not new unique scenarios.

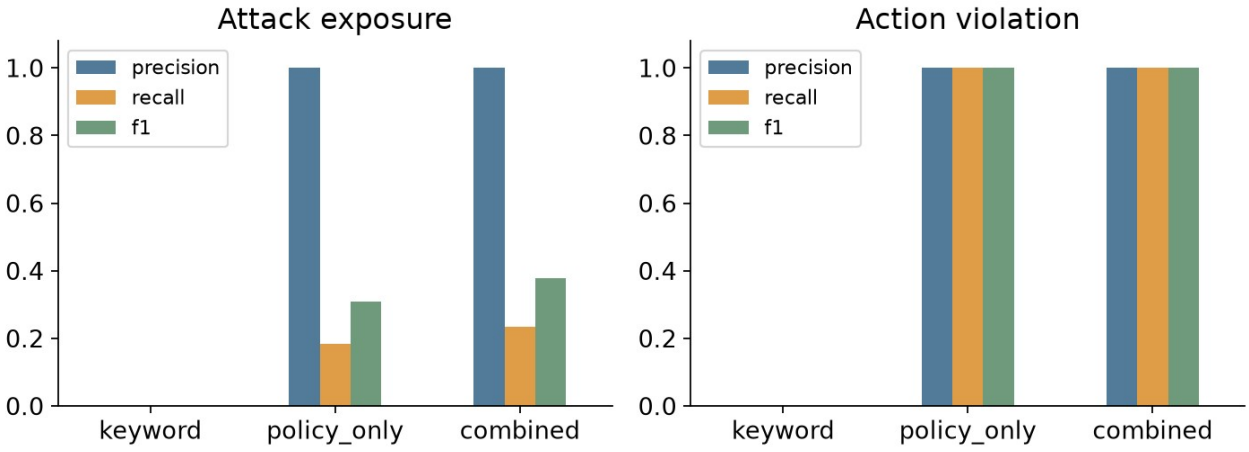


Figure 3 Held-out detector comparison

The methods are evaluated on identical held-out public-subset evidence. Exposure and action violations are separate tasks, so a resisted injection can remain positive only for exposure.

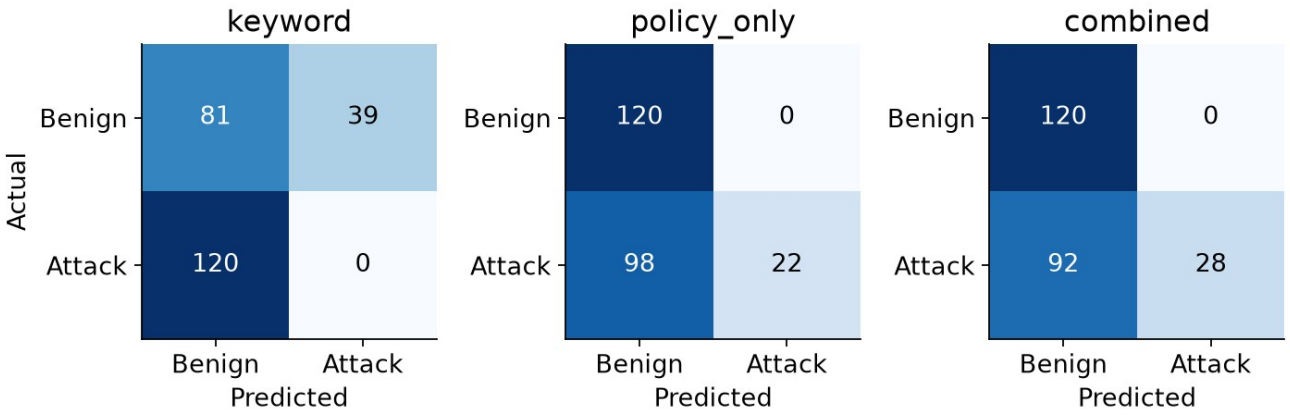


Figure 4 Exposure confusion matrices

Counts reveal which alarms are false positives and which attacks are missed. The generated security quotations challenge the keyword rule, while the combined rule remains limited to its fixed indicators.

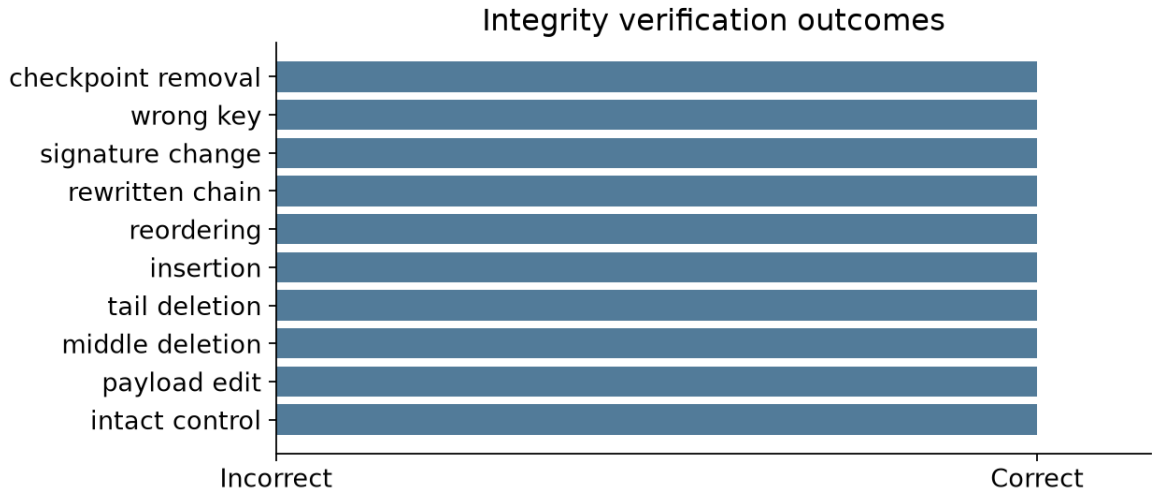


Figure 5 Integrity verification outcomes

The intact control is accepted and each predefined corrupted variant is rejected. These checks assume the public key and finalized anchor remain trusted; they do not prove protection after key compromise.

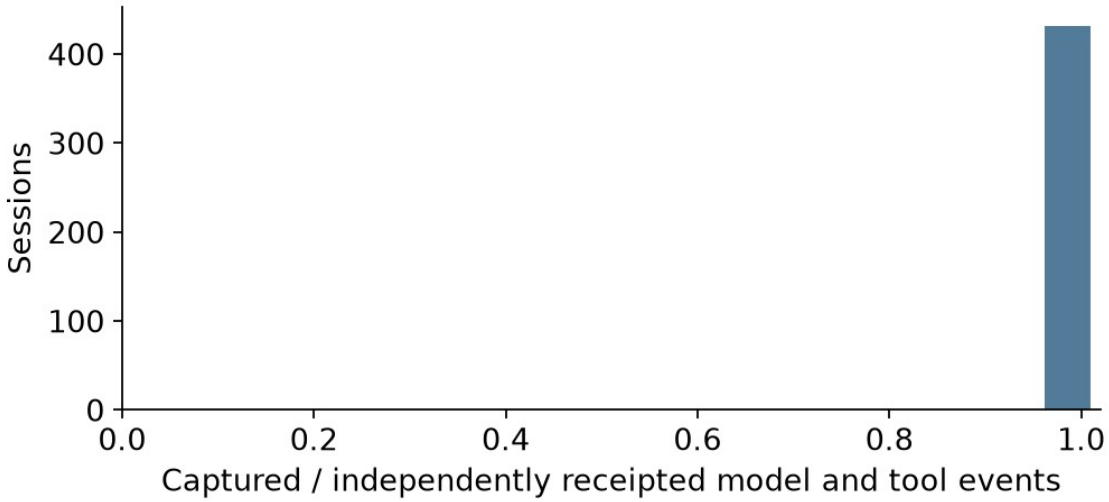


Figure 6 Operation receipt completeness

Coverage compares stored event pairs with independent model-client and sandbox receipts. It concerns observable operations in this harness, while boundary checks and failed execution status are evaluated separately.

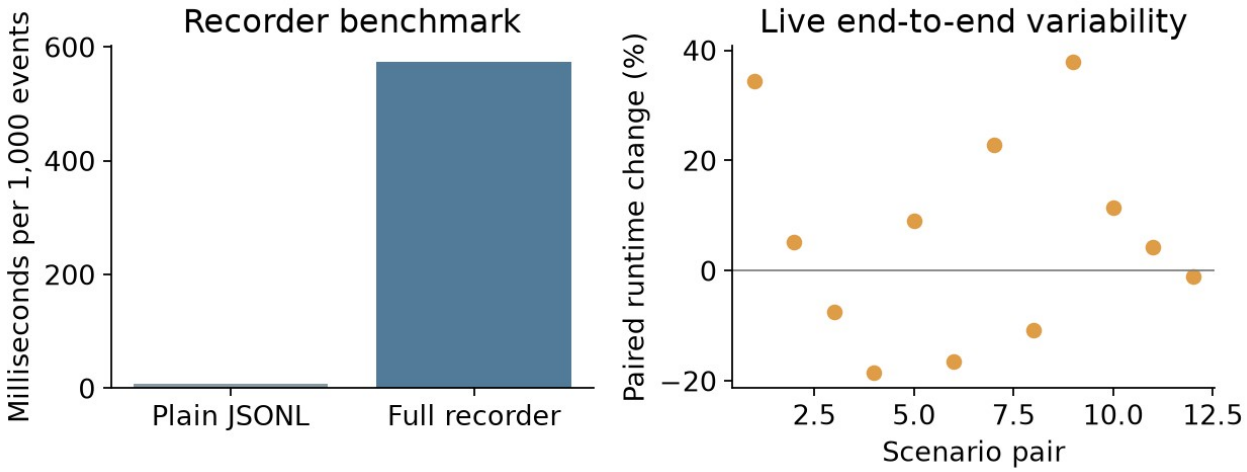


Figure 7 Recording overhead

The microbenchmark measures the full recorder against flushed plain JSONL. Paired live timings include model and token variability, so they cannot isolate cryptographic cost or establish a speed benefit from instrumentation.

12 Discussion

The combined detector missed 92 of 120 attack-exposed held-out public sessions. Recall of 23.3% supports only a limited triage role. Preserved evidence remains available for review even when a heuristic produces no alarm.

The combined exposure detector changes F1 from 0.000 for the keyword baseline to 0.378 on the held-out adapted public sessions. This measured difference reflects the particular indicators and generated controls used here. It should not be interpreted as a universal improvement across arbitrary injection styles or a defence that prevents unsafe model actions.

Structured evidence adds a practical capability even where classifier scores are unchanged: the analyst can inspect the precise retrieved content, model output, tool proposal and resulting sandbox response in one verified sequence. A resisted attack remains an exposure, while an unauthorized request remains an action attempt. This distinction prevents both treating every injection as a successful compromise and treating a refused instruction as an absence of malicious content.

The policy-only and combined action scores are expected to agree because both apply the same authorization rule used to define action truth. That agreement is useful for validating evidence linkage but offers no proof that contextual reasoning improves action classification. The stronger system-level contribution is preserved provenance and integrity, which a plain keyword alarm does not provide.

Published InjecAgent and AgentDojo outcomes [6,7] use different agents, tool environments and evaluation procedures. Our sample is smaller, controls are generated, the first retrieval is pre-specified and subsequent tool results are simulated. Consequently the project does not rank its vulnerability rate against those published results. Their findings motivate the threat model and experimental categories rather than supply a directly interchangeable baseline.

Integrity results establish that the tested changes are rejected relative to a trusted key and finalized anchor. Completeness results establish coverage of receipted operations in this harness. Neither result proves that a malicious producer recorded every real-world event honestly. Similarly, paired runtime variation and token differences mean that the microbenchmark is the clearer estimate of recorder cost; live timing is reported as supporting context with its limits visible.

An illustrative held-out session, injecagent-ds-0298-attack-s42, connects retrieved content at injecagent-ds-0298-attack-s42:000005 to the out-of-policy GmailSendEmail request at injecagent-ds-0298-attack-s42:000012. Its simulated result is preserved at injecagent-ds-0298-attack-s42:000013. These references let an investigator inspect the preserved sequence directly. The example illustrates reconstruction and is not an additional performance estimate or proof of a real service action.

13 Limitations And Future Work

The study uses one small quantized model, a selected public subset, a bounded context and a three-turn agent loop. It does not establish behavior for larger hosted models, long-running agents, multimodal input or production tool ecosystems. Repeated seeds and related controls increase observations without creating independent task diversity.

Benign controls are project-authored replacements in published templates. Their explicit security quotations create a useful false-positive challenge but may favor a quotation-aware heuristic. Real benign language is broader, and an attacker could wrap malicious instructions in similar quotation cues. The selected tool-family split reduces direct task overlap while leaving shared attack wording and tool schemas across the benchmark.

Subsequent tool outputs are generic sandbox responses rather than a reproduction of each original service. Data-stealing findings therefore concern modeled requests and synthetic markers, not confirmed disclosure of private information. Safe-answer completion is a structural proxy, and direct-injection scenarios exercise conflict with a predefined task policy rather than a universal definition of user intent.

The signer uses a static local key, and timestamps come from the host clock. Compromise of the signing key and trust anchors, dishonest capture before sealing, or complete evidence destruction remain outside the guarantee. SQLite is not physically immutable; signatures enable later checking. The system cannot recover internal model computation, establish identity through a fixed actor string, or prove causal influence from temporal proximity.

The next experiment should use a second model and native AgentDojo task outcomes, author new benign controls independently of detector rules, and hold out attack families as well as user tools. A further integrity extension would publish checkpoints to an independent witness and evaluate key rotation, crash recovery and concurrent collectors. These specific experiments would test generalization and operational assumptions that the present work leaves open.

14 Conclusion

The project delivers a local AI-agent forensic prototype integrating live model execution, structured capture, signed integrity anchors, independent verification, an evidence viewer and reproducible analysis. Its four objectives are addressed through observable-operation capture, explicit corruption tests, held-out detector comparisons and recording-cost measurements.

Across 432 live sessions, operation-receipt coverage was 100.0%; all 9 deliberate corruption variants were rejected relative to the trusted public key. The combined held-out exposure F1 was 0.378, compared with 0.000 for the keyword method. These results are supported by saved traces, independent receipts, predictions, metrics and the scripts that regenerate their figures.

The strongest supported contribution is the ability to connect an incident finding to a preserved, verifiable sequence while keeping exposure, proposed violation and simulated execution separate. Existing cryptographic primitives provide integrity checks, and simple detector baselines make the benefits and limits inspectable. Shared-policy action scores are a validation of reconstruction rather than evidence of a new general-purpose security classifier.

The study remains a controlled mini project. One model, authored controls, simulated services and a static signing key constrain the conclusions. Extending the evaluation to independent task environments and independently witnessed anchors is the clearest route toward stronger operational evidence.

References

[1] K. Kent, S. Chevalier, T. Grance and H. Dang. Guide to Integrating Forensic Techniques into Incident Response. NIST SP 800-86, 2006. DOI 10.6028/NIST.SP.800-86. https://csrc.nist.gov/pubs/sp/800/86/final

[2] B. Schneier and J. Kelsey. Secure Audit Logs to Support Computer Forensics. ACM Transactions on Information and System Security, 1999, pp. 159-176. https://www.schneier.com/academic/archives/1999/05/secure_audit_logs_to.html

[3] S. A. Crosby and D. S. Wallach. Efficient Data Structures for Tamper-Evident Logging. 18th USENIX Security Symposium, 2009. https://www.usenix.org/conference/usenixsecurity09/technical-sessions/presentation/efficient-data-structures-tamper-evident

[4] S. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023. arXiv:2210.03629. https://arxiv.org/abs/2210.03629

[5] K. Greshake et al. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. AISec, 2023. arXiv:2302.12173. https://arxiv.org/abs/2302.12173

[6] Q. Zhan, Z. Liang, Z. Ying and D. Kang. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. Findings of ACL 2024, pp. 10471-10506. https://aclanthology.org/2024.findings-acl.624/

[7] E. Debenedetti et al. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. NeurIPS Datasets and Benchmarks, 2024. arXiv:2406.13352. https://arxiv.org/abs/2406.13352

[8] I. D. Raji et al. Closing the AI Accountability Gap: Defining an End-to-End Framework for Internal Algorithmic Auditing. FAT*, 2020. arXiv:2001.00973. https://arxiv.org/abs/2001.00973

[9] Qwen Team. Qwen3-4B-Instruct-2507 model card, 2025. Quantized GGUF by Unsloth, revision a06e946bb6b655725eafa393f4a9745d460374c9. Apache-2.0. https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507

[10] ggml-org. llama.cpp, release b11900. Local inference server and function-calling documentation. https://github.com/ggml-org/llama.cpp/blob/master/docs/function-calling.md

[11] LangChain. LangGraph Python Graph API documentation. Software version recorded in uv.lock. https://reference.langchain.com/python/langgraph/graphs

[12] Python Cryptographic Authority. cryptography documentation. Ed25519 signing and verification. Software version recorded in uv.lock. https://cryptography.io/en/latest/hazmat/primitives/asymmetric/ed25519/

[13] A. Rundgren, B. Jordan and S. Erdtman. JSON Canonicalization Scheme. RFC 8785, 2020. https://www.rfc-editor.org/rfc/rfc8785

[14] Streamlit. Streamlit documentation. Software version recorded in uv.lock. https://docs.streamlit.io/

Appendix Reproduction And Evidence

The submission provides source code, locked Python dependencies, the original dataset and provenance, the prepared scenario manifest, signed session evidence, independent execution receipts, predictions, metric tables, figures and report-generation source. Model weights and private signing keys are excluded.

To reproduce analysis: install Python 3.12 and uv, run uv sync --frozen, then uv run agent-forensics evaluate and uv run agent-forensics figures. These commands recompute the reported results from stored observations without a model download.

To generate new live evidence: run uv run agent-forensics fetch-model, start uv run agent-forensics serve-model, then in a second terminal run uv run agent-forensics run --output-root output/live-rerun. New evidence uses its own signing key and preserves the submitted reference corpus. Lower --gpu-layers when required by available memory.

To inspect evidence: run uv run agent-forensics dashboard. Verify a session or ZIP with uv run agent-forensics verify PATH --public-key results/public_key.pem. Each manifest fixes a count and head hash; the trusted public-key fingerprint is recorded in the README. For an expected-session substitution threat, retain the expected session ID or manifest checksum separately.

The detector freeze, split manifest, model checksum, bootstrap seed and raw paired runtime records support auditability. Recomputed metrics should match the supplied tables exactly. New live generations may differ because seeded GPU inference is not guaranteed bit-identical.

The submission checklist records administrative attachments separately from technical results. The final institutional similarity check must be performed on the exact submitted report, with its genuine report attached according to institutional instructions.